

# **Déploiement d'une application sur les stores**

# Table des matières

|                                                                       |          |
|-----------------------------------------------------------------------|----------|
| <b>I. Contexte</b>                                                    | <b>3</b> |
| <b>II. Généralités et pré-requis au déploiement par environnement</b> | <b>3</b> |
| <b>III. Exercice : Appliquez la notion</b>                            | <b>5</b> |
| <b>IV. Processus de build et magasins d'applications</b>              | <b>6</b> |
| <b>V. Exercice : Appliquez la notion</b>                              | <b>8</b> |
| <b>VI. Essentiel</b>                                                  | <b>9</b> |
| <b>VII. Auto-évaluation</b>                                           | <b>9</b> |
| A. Exercice final.....                                                | 9        |

## I. Contexte

**Durée :** 1 h

**Environnement de travail :** Une application React Native initialisée

**Pré-requis :** Aucun

### Contexte

Une fois que nous avons réalisé notre superbe application, il va falloir la publier sur les magasins applicatifs (appelés *stores* en anglais) afin que les utilisateurs puissent la télécharger.

Cette manipulation nécessite différentes tâches à réaliser qui sont spécifiques à chaque plateforme cible. En effet, ici, il n'y a plus grand-chose lié à React Native : les instructions de déploiement sont exactement les mêmes que pour une application iOS ou Android native.

Dans ce chapitre, nous allons nous intéresser à la théorie autour du déploiement sur les stores. Nous ne serons pas en mesure de montrer toutes les étapes, car nous ne pouvons pas envoyer une application de démo sur les stores Android ou iOS.

## II. Généralités et pré-requis au déploiement par environnement

### Objectifs

- Comprendre la notion d'environnements d'application
- Comprendre le fonctionnement de la signature de paquets applicatifs pour iOS et Android
- Connaître les éléments indispensables pour déployer

### Mise en situation

Avant de pouvoir déployer sur les magasins d'applications tels que l'AppStore, il faut comprendre un certain nombre de principes liés aux déploiements des applications sur Android et iOS. Pour cela, nous allons aborder la notion de configuration d'environnement, ainsi que la notion de signature de paquets applicatifs. Ces procédés étant bien différents selon la plateforme, nous allons expliquer les spécificités d'Android et celles d'iOS.

### Remarque

Dans ce chapitre, tout ce qui est décrit sont des principes généraux liés au déploiement d'applications mobiles. Expo nous simplifie grandement la vie et va automatiser une grande partie des tâches de son côté.

### Le principe d'environnements dans React Native

Lorsque l'on crée une application mobile, il faut directement prendre en compte la notion d'environnement. En effet, lorsque nous développons, il est nécessaire d'activer certains outils de développement et potentiellement de debugger le code. Or, quand l'application est compilée pour la production (l'application finale), il faut que tout soit optimisé pour privilégier la performance : on enlève donc les modules de développement qui ne sont pas utilisés, comme le debugger.

React Native possède par défaut deux environnements : un environnement de développement avec les outils de développement activés, et un environnement de *release* pour les déploiements en production. À chacun de ces environnements est liée une configuration permettant de préparer un build adapté à l'environnement.

- Sur Android, les builds et les environnements sont gérés par **Gradle** : un outil pour gérer des séquences de scripts et de configuration pour les déploiements sur Android.
- Sur iOS, ils sont gérés via **XCode**, qui est l'environnement de développement d'Apple.

Sur iOS, chaque environnement se voit assigner un profil de provision (c'est d'ailleurs à nous de le générer), alors que sur Android, on a besoin d'un **keystore** qui permet de signer le paquet applicatif. Deux technologies différentes, donc, mais qui répondent à la même problématique : certifier l'origine de notre paquet applicatif.

## Qu'est-ce qu'un profil de provision pour iOS ?

Pour signer une application iOS, il faut utiliser un profil de provision (*provisioning profile*). Cet élément est géré par XCode et permet de certifier que l'application réalisée provient bien de notre ordinateur et qu'elle possède les caractéristiques que nous lui avons définies.

En mode développement, il faut également utiliser un autre profil de provision spécifique au développement afin d'autoriser le développeur à tester notre application et à travailler dessus.

Un profil de provision est en fait la somme de 3 choses, si on vulgarise :

- Un certificat de signature, généré via l'interface d'Apple, qui est une sorte de clé publique à laquelle est assignée une clé privée qui nous permet de certifier que c'est la nôtre.
- Un identifiant unique d'application, l'**AppID**, comme `com.masociete.masuperapp`. Sa configuration permet de définir les *capabilities* (une série de permissions qui sont demandées pour utiliser l'application), par exemple la capability *Background localisation*, qui permet d'autoriser le GPS à fonctionner en arrière-plan.
- Et, optionnellement, une liste d'identifiants de terminaux qui pourront installer l'application dans le cas d'une distribution en dehors des stores d'Apple.

Grâce à tout cela, on peut générer un **profil de provision** qui sera utilisé par XCode en fonction de notre environnement cible. Étant donné que tout cela est simplifié par Expo, ce point ne sera pas plus développé ici. On peut également utiliser la configuration automatique de Xcode ou se rendre ici pour en apprendre plus : <https://help.apple.com/xcode/mac/current/#/dev1bf96f17e>.

Dans Expo, les valeurs qui interagissent avec tout cela sont dans le fichier `App.json` et le profil de provision est généré automatiquement pas la CLI (nous verrons cela plus loin).

## Le keystore d'Android

Sur Android, le **keystore** est un fichier semblable, c'est-à-dire une clé privée permettant de certifier la provenance d'un paquet applicatif. En l'utilisant au moment du build, on signe notre application et on certifie donc que l'on est bien l'auteur de cette dernière. La procédure est décrite dans la documentation React Native pour les applications *standalone* <https://reactnative.dev/docs/signed-apk-android>, mais, dans le cas d'Expo, nous pouvons également lui déléguer cette tâche.

Il faut bien garder cette clé, car, pour pouvoir déployer des mises à jour, il faudra signer les nouveaux builds avec cette même clé, sans quoi Google Play refusera l'application.

Là aussi, tout se passe dans `App.json`.

## App.json et Expo

Afin de pouvoir mettre notre application en ligne, il va falloir lui affecter une image qui servira à l'identifier une fois sur le téléphone, mais également un titre.

Couplé à cela et sur chaque store, il faudra saisir une fiche technique expliquant ce qu'est l'application, et proposer une démonstration à l'aide de captures d'écran.

Sur Expo, ces configurations sont abstraites et tout se passe dans le fichier `App.json` à la racine du projet : on voit par exemple la clé `name`, ainsi que `icon` qui permettent de définir le nom de l'application et son icône ; et `version` qui permet de définir la version de l'application. Ces trois champs sont requis.

Viennent ensuite les configurations pour iOS et Android avec le *package name* / *bundle identifier* dont nous avons parlé précédemment. Notez également les notions de `buildNumber` et `versionCode` qui sont équivalentes : elles distinguent les différents binaires de l'application. Il faut s'assurer de les incrémenter pour chaque build que l'on charge sur l'AppStore ou sur la boutique Google Play.

```
1 {
2   "expo": {
3     "name": "Ma super application",
4     "icon": "./chemin/vers/limage-de/app.png",
5     "version": "1.0.0",
6     "slug": "your-app-slug",
7     "ios": {
8       "bundleIdentifier": "com.yourcompany.yourappname",
9       "buildNumber": "1.0.0"
10    },
11    "android": {
12      "package": "com.yourcompany.yourappname",
13      "versionCode": 1
14    }
15  }
16 }
```

La liste exhaustive des configurations est disponible ici : <https://docs.expo.io/versions/latest/config/app/>.

#### Syntaxe À retenir

- Avant de builder une application mobile, il faut comprendre la notion d'environnements d'application. Il existe au minimum deux environnements : celui de développement et celui de production.
- Chaque environnement a sa propre configuration et ses propres modalités de signature du paquet applicatif qu'il représente. C'est une tâche un peu complexe sur une application native classique, mais, grâce à Expo, tout cela est très simple.
- Afin de déployer notre application, il faut également préparer un certains nombre d'éléments : une icône, un écran d'accueil et un lot de détails au sujet de l'application, qui seront demandés par les magasins d'applications afin de créer la fiche technique de l'application.

#### Complément

- <https://reactnative.dev/docs/signed-apk-android>
- <https://docs.expo.io/distribution/app-stores/>

## Exercice : Appliquez la notion

### Exercice

Quel est le nom du fichier dans lequel se trouve la configuration d'Expo ?

- ☐ App.json
- ☐ Expo-config.json
- ☐ main.json

### Exercice

À quoi servent le `versionCode` et `buildIdentifier` ?

- ☐ Ils servent à identifier une temporalité entre potentiellement deux même versions
- ☐ Ils servent à indiquer la version d'Android/iOS minimum requise
- ☐ Ils servent à indiquer la version de React Native à utiliser

Exercice

Qu'est-ce que l'AppID ?

- ☐ L'identifiant du compte Apple utilisé pour signer les paquets
- ☐ L'identifiant de l'application pour signer les paquets

## IV. Processus de build et magasins d'applications

### Objectifs

- Comprendre les principes de base du build natif
- Comprendre les spécificités liées à Expo pour déployer une application sur les stores

### Mise en situation

Lorsque l'on build une application native, c'est un peu complexe, car il faut spécifier bon nombre de configurations par environnement et s'assurer de la compatibilité entre les librairies natives.

En utilisant le **Managed Workflow** d'Expo comme nous l'avons vu dans ce cours, la plupart des concepts sont abstraits, mais traités par Expo en interne : c'est l'utilité de l'outil et l'une de ses principales forces, car cela permet de démocratiser la création d'applications mobiles.

### Généralité de build natif

Avant de voir les spécificités d'Expo, faisons un rapide tour d'horizon du build natif pour comprendre ce que fera Expo à notre place :

- Sur Android, Gradle (le gestionnaire de tâches d'Android) va builder notre application en optimisant le code et, sur une application React Native classique, en copiant le *bundle* JavaScript du packager dans ses *assets*. Il signe également ce paquet applicatif avec notre *key store* pour certifier la provenance de l'archive.
- Sur iOS, XCode fait exactement la même chose en compilant le code Objective-C et en ajoutant le profil de provision à l'application afin de la signer.

Cette opération prend plusieurs minutes en fonction du nombre de dépendances et est nécessaire à chaque nouveau changement.

### Spécificités à Expo

En utilisant Expo, comme nous l'avons vu, la plupart des tâches ne sont pas à notre charge et cela simplifie grandement le processus. Cependant, Expo diffère sur deux points :

- La gestion du *bundle* JavaScript de notre application
- Et la gestion des *assets* statiques (images, vidéos...) que nous utilisons

## Mises à jour à la volée avec du « Code Push »

Sur Expo, au lieu d'embarquer un fichier JavaScript compilé par le packager **Metro Bundle**, on embarque une première version de ce fichier, puis on va aller regarder sur un serveur distant si une version plus récente est disponible. Cette version prendra la priorité si c'est le cas.

Pour cela, Expo, à chaque mise à jour, va générer une nouvelle version du *bundle* JavaScript et l'envoyer sur ce qu'il appelle un canal (*channel* en anglais) de distribution. Cette notion de canaux est très importante, car elle permet de gérer les environnements et les versions intermédiaires à tester avant d'envoyer du code en production.

L'application de production a en quelque sorte un curseur qui pointe vers un canal particulier, nommé par défaut `default` (cependant, on pourra le spécifier au moment du build). C'est très pratique, car on peut tester nos modifications sur un canal, que l'on appelle par exemple `deploiement-du-samedi`, et, une fois que l'on a validé les changements, on peut les pousser sur le canal de notre application en production : les utilisateurs auront ainsi la dernière version à leur prochaine connexion, sans avoir à passer par l'AppStore.

Une fois que l'on souhaite pousser du code sur un canal, il suffit de taper la commande `expo publish --release-channel <nom_du_canal>`, sachant que, si on ne le spécifie pas, cela sera `default`.

On peut accéder au canal courant de l'application sur laquelle on se trouve de cette manière :

```
1 import Constants from 'expo-constants';
2
3 function getEnvironment() {
4   let releaseChannel = Constants.manifest.releaseChannel;
5
6   if (releaseChannel === undefined) { // Si undefined, c'est qu'on est en dev
7     return {envName: "DEVELOPMENT", dbUrl: 'aaa', apiKey: 'bbb'};
8   }
9   if (releaseChannel.indexOf('prod') !== -1) { // Ici on match pour tous les canaux qui
10     commencent par prod, par exemple prod-v1 ou prod-v2
11     return {envName: "PRODUCTION", dbUrl: 'ccc', apiKey: 'ddd'};
12   }
13   if (releaseChannel.indexOf('staging') !== -1) { // Ici pareil mais pour staging, notre
14     environnement de test
15     return {envName: "STAGING", dbUrl: 'eee', apiKey: 'fff'};
16   }
17 }
```

## Assets sur le Cloud via un CDN

Deuxième parti pris : quand le projet est publié, Expo va charger les **assets** sur un CDN (**Amazon CloudFront**) afin qu'ils puissent être récupérés lorsque les utilisateurs lanceront l'application. Il remplace automatiquement dans le code la référence pour pointer vers l'URL sur le CDN.

Toutefois, pour que les **assets** soient chargés sur le CDN, ils doivent être explicitement importés quelque part dans le code de l'application (par exemple, utilisés dans un composant `<Image/>`). S'ils se trouvent dans un import conditionnel, alors le packager ne pourra pas les détecter et ils ne seront donc pas chargés lorsque l'on publiera le projet.

Il est possible de configurer Expo de façon à omettre certains fichiers. Pour cela, il faudra utiliser la configuration suivante dans le fichier `App.js` son :

```
1 "assetBundlePatterns": [
2   "assets/images/*"
3 ],
```

Tout **asset** qui ne se trouverait pas dans le ou les chemins autorisés serait alors omis. Une bonne pratique est d'utiliser la commande `npx expo-optimize` afin d'optimiser tous les fichiers de notre projet avec **sharp**, un programme de traitement d'images, comme décrit ici : <https://docs.expo.io/guides/assets/#optimization>.

## Commandes de build via la CLI et processus d'enchaînement

Maintenant que nous avons défini nos canaux de distribution et tout ce qu'il faut avec, voyons comme builder notre application pour un canal. C'est très simple : il suffit de lancer la commande `EAS Build` ou `expo build:ios` et de se laisser guider par la CLI.

À noter qu'on peut également utiliser le paramètre `--release-channel <canal>` afin de spécifier le canal sur lequel l'application va pointer, sinon le canal sera `default`.

Par exemple, pour iOS : `expo build:ios --release-channel prod-v1`.

Sur Android, nous pourrions choisir le format d'export APK, mais il est conseillé d'utiliser plutôt le format `Android App Bundle` en rajoutant à la commande de build Android l'argument `-t app-bundle`. Cependant, il faut s'assurer que la fonction *Google Play App Signing* est activée pour le projet (pour en savoir plus à ce sujet : <https://developer.android.com/guide/app-bundle>).

S'il manque des clés, la CLI va automatiquement nous guider pour renseigner les champs manquants. Pratique !

### Syntaxe À retenir

- Les builds natifs peuvent être un peu complexes à prendre en main, mais, pour cette initiation, nous utilisons Expo, qui est un outil formidable pour développer une application React Native rapidement. En utilisant les canaux de déploiement, on peut facilement mettre à jour une application à distance en esquivant le procédé de validation des magasins d'applications.
- Enfin, le procédé de build est très simplifié et pris en très grande partie en main par la CLI, qui va nous guider dans la procédure de déploiement.

### Complément

- <https://docs.expo.io/distribution/introduction/>

## Exercice : Appliquez la notion

Exercice

Avec Expo, quelle est l'utilité des channels ?

- ☐ Ils permettent de diffuser une application sur différentes cibles en fonction des versions de celles-ci
- ☐ C'est une configuration de build spécifique

Exercice

Si je ne passe pas de canal de distribution par défaut, que se passe-t-il ?

- ☐ L'application plante
- ☐ Le canal `production` est utilisé
- ☐ Le canal `default` est utilisé

Exercice



Je peux accéder au canal de distribution courant en utilisant...

- ☐ Constants.manifest.releaseChannel
- ☐ Expo.getReleaseChannel()
- ☐ Je ne peux pas y accéder

## VI. Essentiel

Déployer une application requiert quelques considérations en amont : il faut notamment préparer sa fiche descriptive et des captures d'écrans. Il faut également mettre en place un peu de configuration en fonction de la plateforme, notamment pour signer l'application.

La plupart de ces tâches sont réalisées par Expo : c'est un de ses gros avantages, plutôt que de créer une application React Native « *from scratch* ». Expo se charge également de builder notre application sur ses propres serveurs et de gérer le procédé de signature de notre application. Il gère ensuite les nouveaux déploiements en passant outre les stores, grâce à un système de « Code Push ». En développement, on peut utiliser l'application Expo et utiliser le JS que nous écrivons à travers elle.

## VII. Auto-évaluation

### A. Exercice final

#### Exercice 1

Exercice

Combien d'environnements sont présents par défaut dans React Native, sans Expo ?

- ☐ 1
- ☐ 2
- ☐ 3
- ☐ 4

Exercice

Quel outil gère les builds de code sur Android ?

- ☐ XCode
- ☐ Puppeteer
- ☐ Gradle
- ☐ Android Studio

Exercice

Quel outil gère les builds de code sur iOS ?

- ☐ Gradle
- ☐ Puppeteer
- ☐ iOS Studio
- ☐ XCode

Exercice

Qu'est-ce qu'un profil de provision ?

- ☐ Un profil de performance pour tester la réactivité d'une application
- ☐ Un profil de paiement pour provisionner des fonds sur son compte afin de payer son abonnement annuel à l'AppStore
- ☐ Un regroupement d'informations permettant de signer une application iOS

Exercice

Qu'est-ce qu'un keystore Android ?

- ☐ C'est une chaîne d'actions définie dans Gradle pour réaliser les différentes étapes de la compilation d'une application Android
- ☐ C'est une clé privée permettant de signer l'application Android avant de la déployer
- ☐ Une liste de clés/valeurs pour des données sécurisées stockée dans le terminal Android de l'utilisateur

Exercice

Si je perd mon keystore, je peux envoyer les nouvelles mises à jour de mon application sur Google Play.

- ☐ Vrai
- ☐ Faux

Exercice

Parmi ces champs du fichier App . json, lesquels sont obligatoires ?

- ☐ icon
- ☐ slug
- ☐ name
- ☐ version

Exercice

Pour gérer les différentes versions du *bundle* JavaScript, Expo utilise...

- ☐ Des canaux de distribution
- ☐ Les versions de l'application

Exercice

Expo héberge les *assets* de l'application sur un CDN, mais lequel ?

- ☐ Cloud Content Delivery Network
- ☐ Amazon Cloudfront
- ☐ Azure Content Delivery Network

Exercice

Pour empêcher qu'Expo n'envoie les `assets` sur le CDN, mais les garde plutôt en local, j'utilise...

- ☐ `assetBundlePatterns` dans `App.json`
- ☐ `offlineAssetsPatterns` dans `App.json`
- ☐ Ce n'est pas possible

